DSA Experiment 8:Implement Binary search tree with operations Create, search, and recursive traversal.

Name:Rushita Kiran Sontakke **PRN:B25ET1051**
Batch:B1 **Class:SY1**

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *left, *right;
};


struct Node* createNode(int value) {
    struct Node* newNode =
        (struct Node*)malloc(sizeof(struct Node));

    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }

    newNode->data = value;
    newNode->left = NULL;
    newNode->right = NULL;

    return newNode;
}


struct Node* create(struct Node* root, int value) {
    if (root == NULL) {
        return createNode(value);
    }

    if (value < root->data) {
        root->left = create(root->left, value);
    }
    else if (value > root->data) {
        root->right = create(root->right, value);
    }
```

```
    return root;
}


struct Node* search(struct Node* root, int key) {
    if (root == NULL || root->data == key) {
        return root;
    }

    if (key < root->data) {
        return search(root->left, key);
    }

    return search(root->right, key);
}


void inorder(struct Node* root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}


void preorder(struct Node* root) {
    if (root != NULL) {
        printf("%d ", root->data);
        preorder(root->left);
        preorder(root->right);
    }
}


void postorder(struct Node* root) {
    if (root != NULL) {
        postorder(root->left);
        postorder(root->right);
        printf("%d ", root->data);
    }
}
```

```
int main() {
    struct Node* root = NULL;
    struct Node* result;
    int choice, value, n, i;

    do {
        printf("\n\n--- Binary Search Tree ---");
        printf("\n1. Create ");
        printf("\n2. Search");
        printf("\n3. Inorder Traversal");
        printf("\n4. Preorder Traversal");
        printf("\n5. Postorder Traversal");
        printf("\n6. Exit");
        printf("\nEnter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter node ");
                scanf("%d", &n);

                for (i = 0; i < n; i++) {
                    printf("Enter value: ");
                    scanf("%d", &value);
                    root = create(root, value);
                }
                break;

            case 2:
                printf("Enter value to search: ");
                scanf("%d", &value);

                result = search(root, value);

                if (result != NULL)
                    printf("Element found!");
                else
                    printf("Element not found!");
                break;

            case 3:
                printf("Inorder: ");
                inorder(root);
                break;
```

```
            case 4:
                printf("Preorder: ");
                preorder(root);
                break;

            case 5:
                printf("Postorder: ");
                postorder(root);
                break;

            case 6:
                printf("Exiting...");
                break;

            default:
                printf("Invalid choice!");
        }
    } while (choice != 6);

    return 0;
}
```

Output

```
--- Binary Search Tree ---
1. Create
2. Search
3. Inorder Traversal
4. Preorder Traversal
5. Postorder Traversal
6. Exit
Enter your choice: 1
Enter node 4
Enter value: 32
Enter value: 12
Enter value: 45
Enter value: 98
```

Output

```
--- Binary Search Tree ---
1. Create
2. Search
3. Inorder Traversal
4. Preorder Traversal
5. Postorder Traversal
6. Exit
Enter your choice: 2
Enter value to search: 98
Element found!

--- Binary Search Tree ---
1. Create
2. Search
3. Inorder Traversal
4. Preorder Traversal
5. Postorder Traversal
6. Exit
Enter your choice: 3
Inorder: 12 32 45 98
```

Output

```
--- Binary Search Tree ---
1. Create
2. Search
3. Inorder Traversal
4. Preorder Traversal
5. Postorder Traversal
6. Exit
Enter your choice: 4
Preorder: 32 12 45 98

--- Binary Search Tree ---
1. Create
2. Search
3. Inorder Traversal
4. Preorder Traversal
5. Postorder Traversal
6. Exit
Enter your choice: 5
Postorder: 12 98 45 32
```

```
--- Binary Search Tree ---
1. Create
2. Search
3. Inorder Traversal
4. Preorder Traversal
5. Postorder Traversal
6. Exit
Enter your choice: 6
Exiting...
```